SmartTask

Plateforme Logistique Intelligente

Rapport Complet du Projet PFE

Projet de Fin d'Etudes (PFE)

Date : 01/09/2026

Ce document presente l'ensemble du projet SmartTask : problematique, solution, methodologie, architecture, technologies, diagrammes UML, backend, frontend, microservice IA et chaine d'automation.

Table des matieres

1. Introduction et contexte
2. Problematique
3. Solution proposee
4. Methodologie de developpement
5. Architecture globale
6. Technologies et langages utilises
7. Diagramme de cas d'utilisation
8. Diagramme de classes
9. Diagrammes de sequence
10. Base de donnees
11. Backend (Spring Boot) - detail
12. Frontend (Angular) - detail
13. Microservice IA (Python FastAPI)
14. Chaine d'automation IA
15. Securite et gestion des roles
16. Tests et validation
17. Guide de demarrage
18. Conclusion

1. Introduction et contexte

SmartTask est une plateforme web de gestion logistique destinee aux entreprises de livraison. Elle permet de gerer les commandes, les taches de livraison, les chauffeurs, les clients et les incidents, tout en integrant l'intelligence artificielle pour predire les retards, recommander les chauffeurs et automatiser les livraisons express.

Le projet est developpe selon une architecture en trois couches : une interface Angular, un backend Java Spring Boot et un microservice Python dedie a l'IA.

2. Problematique

Les entreprises de logistique font face a plusieurs defis majeurs dans la gestion quotidienne de leurs operations :

- Traitement manuel des commandes : validation, preparation et assignation prennent du temps.
- Manque de visibilite : managers et clients ne suivent pas facilement l'etat des livraisons.
- Assignation non optimale : choix du chauffeur base sur l'experience, pas sur des donnees.
- Retards non anticipes : pas d'alerte precoce avant qu'un retard ne se produise.
- Communication fragmentee : pas d'assistant centralise pour repondre aux questions.
- Typage des produits : saisie libre des produits sans standardisation.

Question centrale : Comment concevoir une plateforme intelligente qui automatise les livraisons urgentes, assiste les decideurs par l'IA et offre une experience multi-acteurs (admin, manager, operateur, chauffeur, client) ?

3. Solution proposee

SmartTask repond a la problematique par les modules suivants :

Module	Description
Gestion multi-roles	5 espaces dedies : Admin, Manager, Operateur, Chauffeur, Cli
Cycle commande/livraison	Workflow complet de la commande a la reception client
Types de produits	Liste standardisee + option Autre pour saisie libre
IA predictive	Retard, ETA, priorisation, recommandation chauffeur
Automation express	Chaine autonome 7 etapes pour commandes HIGH/CRITICAL
Chatbot Copilot	Assistant contextuel par role avec historique
Incidents & avis	Signalement et evaluation post-livraison
Notifications	Alertes temps reel pour tous les acteurs

4. Methodologie de developpement

Approche adoptee

Developpement iteratif et incremental, organise en phases fonctionnelles :

- Phase 1 : Authentification JWT et gestion utilisateurs (5 roles)
- Phase 2 : Entites metier (commandes, taches, chauffeurs, notifications)
- Phase 3 : Interfaces Angular par role avec design system
- Phase 4 : Microservice Python IA (predictions ML)
- Phase 5 : Integration Spring Boot <-> Python via WebClient
- Phase 6 : Chatbot SmartTask Copilot
- Phase 7 : Chaine Livraison Express Autonome (automation)
- Phase 8 : Types de produits avec option Autre

Outils

- IDE : Cursor / IntelliJ / VS Code
- Versionnement : Git
- Migrations BDD : Flyway (V1 a V10)
- API docs : Swagger OpenAPI (SpringDoc)
- Tests : JUnit 5 + Spring Boot Test

5. Architecture globale

Architecture 3 tiers

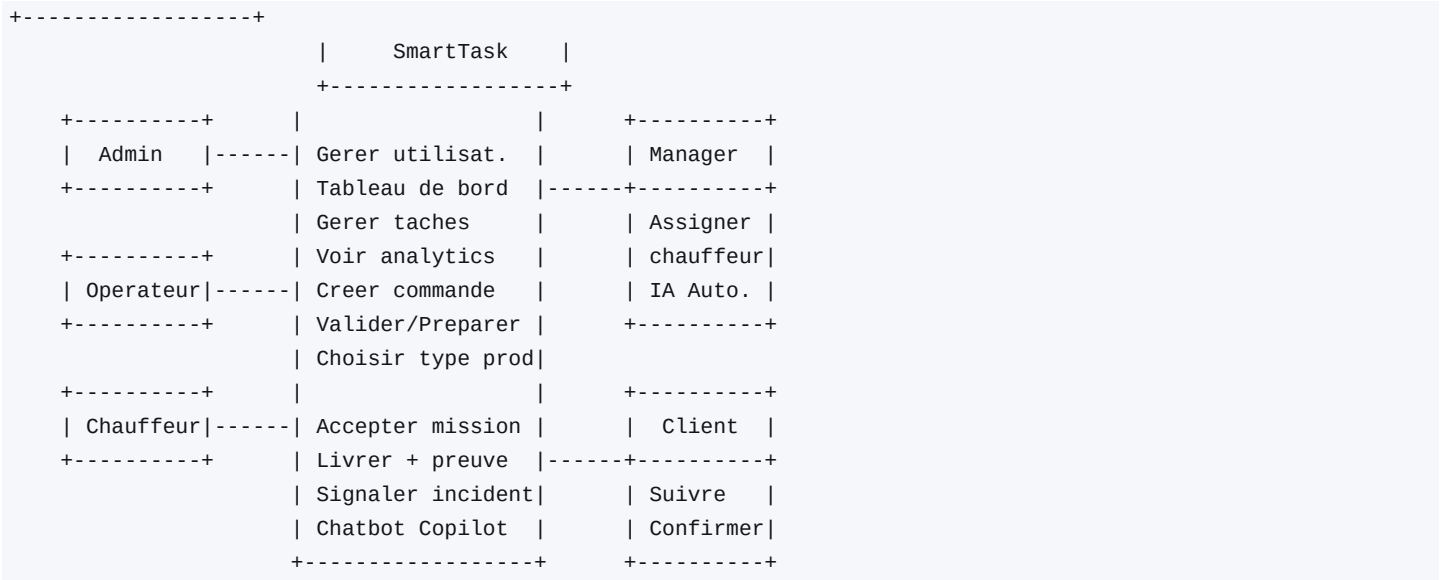
Principe cle : le frontend Angular ne communique JAMAIS directement avec Python. Toutes les requetes passent par Spring Boot qui orchestre la logique metier et delegue les calculs IA au microservice Python.

6. Technologies et langages utilises

Couche	Technologie	Version	Role
Frontend	Angular + TypeScript	16.2	Interface utilisateur SPA
Frontend	RxJS	7.8	Programmation reactive
Backend	Java + Spring Boot	17 / 4.x	API REST, securite, metier
Backend	Spring Security + JWT	jjwt 0.12	Authentification stateless
Backend	Spring Data JPA	-	ORM Hibernate
Backend	Flyway	-	Migrations SQL versionnees
IA	Python + FastAPI	3.11+	Microservice predictions ML
IA	scikit-learn / XGBoost	-	Modeles ML
BDD	PostgreSQL	14+	Stockage persistant
Build	Gradle / npm	-	Compilation et dependances

7. Diagramme de cas d'utilisation

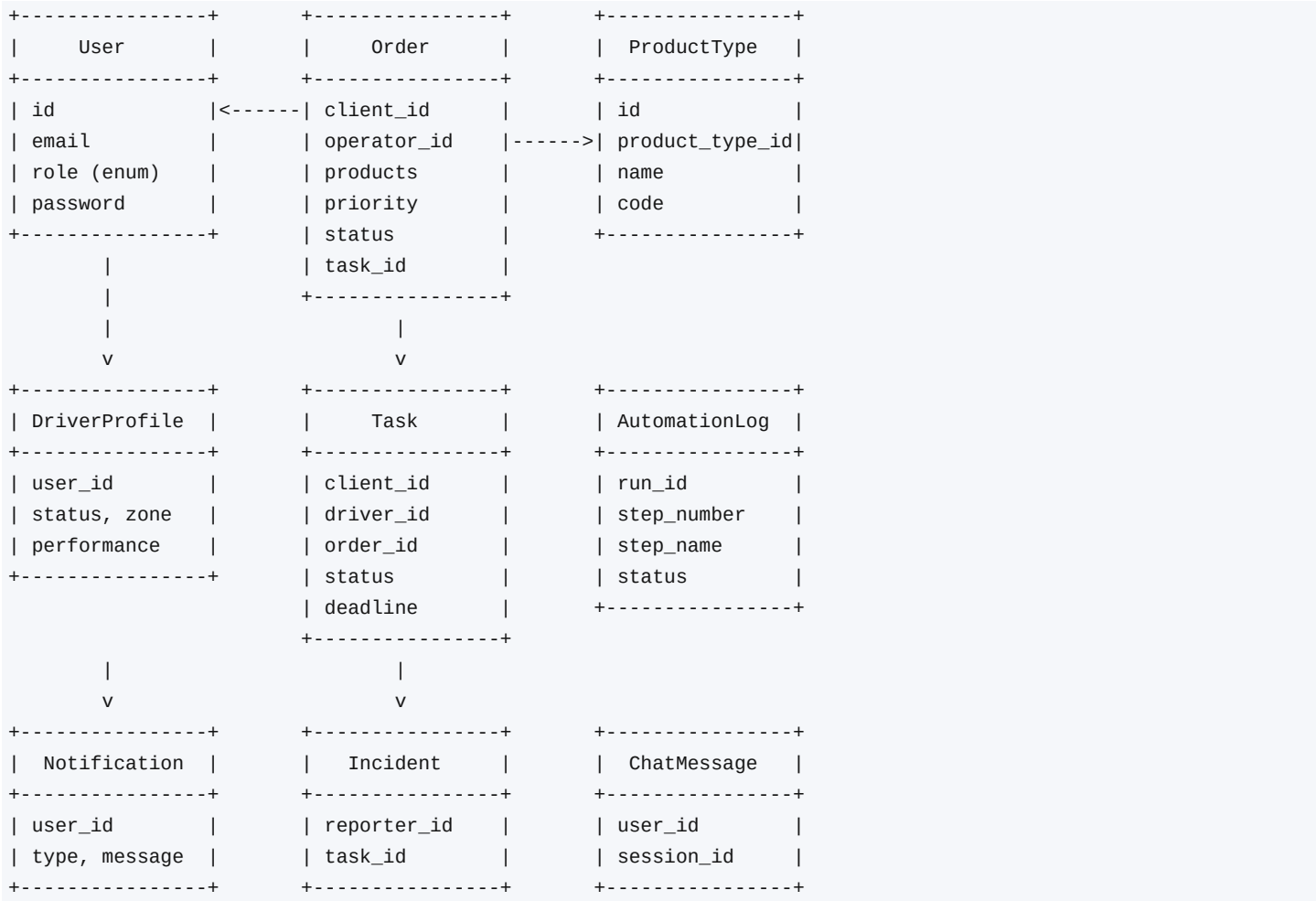
Acteurs et cas d'utilisation principaux



Acteur	Cas d'utilisation principaux
Admin	CRUD utilisateurs, activer/desactiver, changer roles
Manager	Dashboard KPIs, taches, chauffeurs, retards, IA, automation
Operateur	Creer commande (type produit), valider, preparer, incidents
Chauffeur	Accepter mission, demarrer, livrer, preuve, GPS
Client	Voir commandes, suivi livraison SSE, confirmer, avis
Systeme IA	Predire retard, ETA, prioriser, recommander chauffeur
Automation	Chaine express 7 etapes (auto HIGH/CRITICAL)

8. Diagramme de classes (simplifie)

Entites principales et relations



Enums principaux : Role, TaskStatus, OrderStatus, Priority, DriverStatus, IncidentStatus

9. Diagrammes de sequence

9.1 Authentification

Sequence - Login JWT

```
Utilisateur    Angular         Spring Boot       PostgreSQL
    |              |                 |                |
    |-- login ---->|                 |                |
    |              |-- POST /auth/login ------------->|
    |              |                 |-- verify pwd ->|
    |              |                 |<-- user -------|
    |              |<-- JWT token ---|                |
    |<-- redirect -|                 |                |
    |              |-- requetes + Bearer token ------->|
```

9.2 Creation commande + Automation

Sequence - Commande HIGH + Chaine IA

```
Operateur   Angular    Spring Boot    EventListener   Python IA    PostgreSQL
   |           |            |               |             |            |
   |-- creer ->|            |               |             |            |
   |           |-- POST order ----------->  |             |            |
   |           |            |-- save ------------------------------>|
   |           |            |-- publish OrderCreatedEvent ->|        |
   |           |            |               |-- execute chain ----->|
   |           |            |               |-- AUTO_VERIFY ------->|
   |           |            |               |-- CREATE_TASK ------->|
   |           |            |               |-- AI_PRIORITIZE ----->|
   |           |            |               |<-- score -------------|
   |           |            |               |-- AI_ASSIGN_DRIVER -->|
   |           |            |               |-- NOTIFY client ---->|
   |           |            |               |-- LOG_COMPLETE ----->|
   |<- OK -----|<-----------|               |             |            |
```

9.3 Prediction IA

Sequence - Prediction retard

```
Manager    Angular    Spring Boot    SmartTaskAIClient    Python IA
  |          |            |                 |                |
  |-- click->|            |                 |                |
  |          |-- POST predict-delay ------> |                |
  |          |            |-- WebClient POST /predict/delay ->|
  |          |            |                 |<-- ML result --|
  |          |<-- JSON ---|<----------------|                |
  |<- affiche|            |                 |                |
```

10. Base de donnees

PostgreSQL avec 10 migrations Flyway (V1 a V10) :

Migration	Contenu
V1	Table users (5 roles)
V2	driver_profiles, tasks, task_history, notifications
V3	orders, incidents, reviews
V4	Statut RECEIVED pour commandes
V5	Seed commande DELIVERED pour tests client
V6	Index de performance
V7	Preuve de livraison (image)
V8	chat_messages (chatbot)
V9	automation_logs (chaine IA)
V10	product_types + lien orders

11. Backend (Spring Boot) - Detail

Structure des packages

- controller/ : 12 controleurs REST (Auth, User, Task, Manager, Operator, Driver, Client, Automation, Chat, ProductType...)
- service/ : logique metier (OperatorService, TaskService, ManagerService...)
- service/automation/ : ExpressDeliveryAutomationService (7 etapes)
- service/chat/ : ChatService, ChatContextBuilder, ChatFallback
- service/ai/ : CompositeAIRecommendationService (Python + fallback regles)
- entity/ : 11 entites JPA + enums
- security/ : JwtAuthenticationFilter, CustomUserDetailsService
- event/ : OrderCreatedEvent + AutomationEventListener (@Async)

Controleurs principaux

Controller	Prefixe	Role
AuthenticationController	/api/auth	Public
UserController	/api/users	ADMIN
TaskController	/api/tasks	MANAGER
ManagerController	/api/manager	MANAGER
OperatorController	/api/operator	OPERATOR
DriverController	/api/driver	DRIVER
ClientController	/api/client	CLIENT
AutomationController	/api/automation	MANAGER/OPERATOR
ChatController	/api/chat	Authentifie
ProductTypeController	/api/product-types	Authentifie

Cycle de vie commande

PENDING -> VERIFIED -> PREPARING -> READY -> IN_DELIVERY -> DELIVERED -> RECEIVED

Cycle de vie tache

PENDING/PREPARED -> ASSIGNED -> ACCEPTED -> IN_PROGRESS -> IN_DELIVERY -> COMPLETED

12. Frontend (Angular) - Detail

Organisation

- core/ : auth (guard, interceptor JWT), services HTTP, modeles TypeScript
- features/ : composants par role (admin, manager, operator, driver, client)
- shared/ : chat-widget, design system, styles globaux
- Routes lazy-loaded pour performance

Pages par role

Role	Pages principales
Manager	Dashboard, Taches, Planning, Carte, Chauffeurs, Retards, IA,
Operateur	Dashboard, Commandes, Nouvelle commande (types produit), Tac
Chauffeur	Dashboard, Missions, Route, Historique, Performance
Client	Dashboard, Commandes, Suivi livraison, Avis, Notifications
Admin	Dashboard, Gestion utilisateurs

Services Angular cles

- AuthService : login/logout, stockage JWT localStorage
- AutomationService : status, logs, lancer chaine express
- ProductTypeService : liste types de produits
- ChatService : init session, envoyer message, historique
- AiService : predictions via backend (pas d'appel direct Python)

13. Microservice IA (Python FastAPI)

Microservice independant sur le port 8000. Spring Boot l'appelle via WebClient.

Endpoint	Description
GET /api/ai/health	Sante du service
POST /api/ai/predict/delay	Prediction risque de retard
POST /api/ai/predict/delivery-time	Estimation ETA (minutes)
POST /api/ai/prioritize/task	Score de priorite
POST /api/ai/recommend/driver	Recommandation chauffeur
POST /api/ai/detect/anomaly	Detection anomalies
POST /api/ai/chat	Reponse chatbot Copilot

Modeles ML

- delay_model.pkl : classification retard (Logistic Regression / Random Forest / XGBoost)
- delivery_time_model.pkl : regression temps livraison (XGBoost)
- Fallback rule-based si Python indisponible

14. Chaine d'automation IA

La chaine « Livraison Express Autonome » execute 7 instructions automatiquement lorsqu'une commande HIGH ou CRITICAL est creee.

Etape	Nom	Action
1	READ_ORDER	Lire les details de la commande
2	AUTO_VERIFY	Valider automatiquement (PENDING -> VERIFIED)
3	CREATE_TASK	Creer tache PREPARED, commande -> READY
4	AI_PRIORITIZE	Prioriser via microservice Python
5	AI_ASSIGN_DRIVER	Assigner chauffeur si score >= 70%
6	ESTIMATE_AND_NOTIFY	Calculer ETA + notifier client/chauffeur
7	LOG_COMPLETE	Ecrire journal dans automation_logs

Declenchement

- Automatique : OrderCreatedEvent + priorite HIGH/CRITICAL
- Manuel : POST /api/automation/express/{orderId}
- Visualisation : page Manager > IA Automation

15. Securite et gestion des roles

- Authentification JWT stateless (24h expiration)
- Mots de passe hashes BCrypt
- 5 roles : ADMIN, MANAGER, OPERATOR, DRIVER, CLIENT
- Controle d'acces par URL (@PreAuthorize + SecurityFilterChain)
- Frontend : authGuard + roleGuard + jwt.interceptor

Comptes de test

Email	Mot de passe	Role
admin@smarttask.com	Admin123!	ADMIN
manager@smarttask.com	Manager123!	MANAGER
operator1@smarttask.com	Operator123!	OPERATOR
driver1@smarttask.com	Driver123!	DRIVER
client1@smarttask.com	Client123!	CLIENT

16. Tests et validation

- Tests JUnit : AuthenticationController, ManagerAI, AutomationController, Security
- Profil test : H2 in-memory, Flyway desactive
- Build Angular : ng build (production OK)
- Build Backend : gradlew compileJava / test

Scenarios de test manuels

- Login par role et verification acces pages
- Creer commande avec type produit + option Autre
- Creer commande HIGH -> verifier chaine automation (7 logs)
- Manager : assigner chauffeur, voir analytics
- Chauffeur : accepter et completer mission
- Client : confirmer reception et laisser avis

17. Guide de demarrage

Service	Commande	Port
PostgreSQL	Demarrer service + creer smarttask_db	5432
Backend	gradlew bootRun	9090
Python IA	uvicorn app.main:app --port 8000	8000
Frontend	ng serve	4200

URLs : Frontend http://localhost:4200 | Swagger http://localhost:9090/swagger-ui.html | IA http://localhost:8000/docs

18. Conclusion

SmartTask est une plateforme logistique complete integrant gestion multi-acteurs, intelligence artificielle et automation. Le projet demontre la maitrise d'une architecture microservices (Angular + Spring Boot + Python), la securite JWT, les migrations de base de donnees, et l'automatisation metier via la chaine Livraison Express Autonome.

Perspectives d'evolution : chaines automation supplementaires (gardien anti-retard, recovery incident), deploiement Docker, application mobile chauffeur, et apprentissage ML sur donnees reelles.